Universidad Nacional Autónoma de México

Facultad de Ingeniería — División de Ingeniería Eléctrica

Diseño Arquitectónico y Guía de Implementación: Parser & SDT

Asignatura: Compiladores | **Grupo:** 5 | **Equipo:** 02

Paquete: `unam.fi.compilers.g5.02` | **Fecha límite:** 10 de noviembre de 2026

---

1 Justificación Arquitectónica y Enfoque Teórico

El presente documento define el diseño formal del Analizador Sintáctico (**Parser**) y del esquema de Traducción Dirigida por Sintaxis (**SDT**) para el compilador del Equipo 02. En apego a las especificaciones de evaluación de la asignatura, se adopta la construcción manual basada exclusivamente en los conceptos teóricos vistos en clase (lo cual otorga puntuación adicional sobre la evaluación total), descartando el uso de generadores de caja negra como PLY, SLY, YACC o Bison.

1.1 Selección del Método de Parsing: Predictivo Top-Down LL(1) con Pila Explícita

A partir de la taxonomía de analizadores sintácticos estudiada en las sesiones teóricas, se seleccionó un **Parser Predictivo Descendente LL(1) sin retroceso (*without backtracking*)**, implementado mediante una **Pila Explícita (*Stack*)**, un **Búfer de Entrada (*i/p Buffer*)** y una **Tabla de Análisis Sintáctico (*LL(1) Parsing Table*)**. Esta decisión se fundamenta en tres criterios técnicos:

1. **Trazabilidad Teórica para el Reporte y Defensa Oral:** A diferencia del método de descenso recursivo (que oculta el estado de derivación en la pila de llamadas interna de Python), la arquitectura con pila explícita permite imprimir y auditar en cada iteración la cuádrupla $(\text{Step}, \text{Stack}, \text{Input}, \text{Action})$. Esto proporciona la argumentación teórica exacta exigida para cada salida del reporte técnico.
2. **Ventaja frente a Analizadores Bottom-Up (LR(0), SLR(1), CLR(1), LALR(1)):** Los métodos ascendentes requieren construir la colección canónica de ítems LR mediante las operaciones `closure` y `goto`. Al incorporar el catálogo de palabras reservadas y tipos de datos de Solidity, el autómata de estados LR crece considerablemente y puede presentar conflictos *shift/reduce*. El modelo LL(1) opera directamente sobre los conjuntos $\text{FIRST}()$ y $\text{FOLLOW}()$ con complejidad temporal y espacial lineal.
3. **Desacoplamiento Estructural entre Sintaxis y Semántica (Procedimiento SDT):** Siguiendo el procedimiento de dos pasos establecido en los apuntes de *Syntax Directed Translation*:
   - **Paso 1:** El motor LL(1) valida la estructura gramatical de los tokens emitidos por el Lexer y construye el Árbol de Análisis Sintáctico (*Parse Tree*). Si la pila alcanza el símbolo de fin de entrada `$` al mismo tiempo que el búfer, se garantiza la validez sintáctica (`Parsing Success!`).
   - **Paso 2:** Se realiza un recorrido (*traversal*) sobre el árbol construido ejecutando las acciones semánticas asociadas a cada regla de producción para validar tipos y declaraciones en la Tabla de Símbolos, logrando separar limpiamente un éxito sintáctico de una falla semántica (`Parsing Success!` seguido de SDT `error...`).

2 Arquitectura del Paquete (`unam.fi.compilers.g5.02`)

Siguiendo buenas prácticas de Ingeniería de Software, el paquete existente se extiende incorporando los módulos de análisis sintáctico, árbol sintáctico y verificador SDT, manteniendo el desacoplamiento

por capas:

```
unam/
\-- fi/
    \-- compilers/
        \-- g5/
            \-- 02/
                |-- __init__.py
                |-- main.py              # Orquestador: conecta Lexer -> Parser -> SDT
                |-- io/
                |   |-- reader.py        # Carga de archivo o cadena fuente en memoria
                |   \-- writer.py        # Salida estandarizada y reporte de linea de e
                |-- lexer/
                |   |-- scanner.py       # Generador del flujo de tokens y Panic-
Mode
                |   \-- dfa.py           # Automata Finito Determinista (State Pattern)
                |-- parser/
                |   |-- ll1_parser.py    # Motor LL(1): Pila explicita, Buffer y Tabla
                |   \-- grammar.py       # Producciones CFG, conjuntos FIRST, FOLLOW y
                |-- sdt/
                |   \-- analyzer.py      # Recorrido del Parse Tree y reglas semanticas
                |-- symbol_table/
                |   \-- symtab.py        # Operaciones Insert(), Lookup(), Set() y Rese
                \-- models/
                    |-- token.py         # Objeto Token <Category, Lexeme, Line>
                    \-- ast_node.py      # Nodo del Parse Tree con atributos sintetizad
```

3 Diseño de la Gramática Libre de Contexto (CFG) y Tabla LL(1)

El analizador sintáctico recibe como entrada el flujo de tokens generado por el analizador léxico, clasificado en las siete categorías del proyecto: `Keyword`, `Identifier`, `Operator`, `Constants`, `Literals`, `Punctuators` y `Special_Characters` (donde `$` representa el fin de entrada EOF).

3.1 Eliminación de Recursividad por la Izquierda y Factorización

La gramática aritmética clásica $E \to E + T \mid T$ y $T \to T * F \mid F$ presenta recursividad inmediata por la izquierda ($A \to A\alpha \mid \beta$), lo cual provocaría un bucle infinito en un analizador Top-Down. Aplicando la regla de transformación a recursividad por la derecha vista en clase ($A \to \beta A'$ y $A' \to \alpha A' \mid \epsilon$), e integrando tanto declaraciones con tipo (como el caso obligatorio `int $x=10`) como sentencias de asignación y punto y coma opcional (S_{end}), se define la siguiente CFG determinista:

1. $S \to$ Keyword Identifier $=$ $E\ S_{\text{end}}$
2. $S \to$ Identifier $=$ $E\ S_{\text{end}}$
3. $S_{\text{end}} \to ; \mid \epsilon$
4. $E \to T\ E'$
5. $E' \to +\ T\ E' \mid -\ T\ E' \mid \epsilon$
6. $T \to F\ T'$
7. $T' \to *\ F\ T' \mid /\ F\ T' \mid \epsilon$
8. $F \to (\ E\ ) \mid$ Identifier $\mid$ Constants $\mid$ Literals

3.2 Conjuntos FIRST() y FOLLOW()

Aplicando las tres reglas teóricas para el cálculo de conjuntos:

- **Regla 1:** El símbolo terminal inmediato se selecciona en FIRST, o como FOLLOW si sigue a un no-terminal.
- **Regla 2:** El conjunto FIRST del siguiente no-terminal se agrega al FOLLOW del no-terminal actual.
- **Regla 3:** Si el no-terminal se ubica en el extremo derecho del lado derecho (RHS) o deriva en ϵ, hereda el FOLLOW del símbolo del lado izquierdo (LHS).

No-Terminal	Conjunto FIRST	Conjunto FOLLOW
S	$\{\text{Keyword}, \text{Identifier}\}$	$\{\$\}$
S_{end}	$\{;, \epsilon\}$	$\{\$\}$
E	$\{(, \text{Identifier}, \text{Constants}, \text{Literals}\}$	$\{;,), \$\}$
E'	$\{+, -, \epsilon\}$	$\{;,), \$\}$
T	$\{(, \text{Identifier}, \text{Constants}, \text{Literals}\}$	$\{+, -, ;,), \$\}$
T'	$\{*, /, \epsilon\}$	$\{+, -, ;,), \$\}$
F	$\{(, \text{Identifier}, \text{Constants}, \text{Literals}\}$	$\{*, /, +, -, ;,), \$\}$

Cuadro 1: Conjuntos FIRST y FOLLOW calculados para la gramática del proyecto.

3.3 Construcción de la Tabla de Parsing LL(1)

De acuerdo con las reglas de construcción vistas en clase: (1) todas las producciones ϵ se colocan bajo las columnas de su conjunto FOLLOW, y (2) las producciones restantes se colocan bajo las columnas de su conjunto FIRST. Los huecos vacíos representan errores sintácticos.

NT	Keyword	Identifier (id)	const / lit	=	+ / -	* / /	(	)	; / $
S	$S \to$ Kw id $= ES_{\text{end}}$	$S \to$ id $= ES_{\text{end}}$	—	—	—	—	—	—	—
S_{end}	—	—	—	—	—	—	—	—	$; : S_{\text{end}} \to ; \$: S_{\text{end}} \to \epsilon$
E	—	$E \to TE'$	$E \to TE'$	—	—	—	$E \to TE'$	—	—
E'	—	—	—	—	$E' \to \pm TE'$	—	—	$E' \to \epsilon$	$E' \to \epsilon$
T	—	$T \to FT'$	$T \to FT'$	—	—	—	$T \to FT'$	—	—
T'	—	—	—	—	$T' \to \epsilon$	$T' \to */FT'$	—	$T' \to \epsilon$	$T' \to \epsilon$
F	—	$F \to$ id	$F \to$ const | lit	—	—	—	$F \to (E)$	—	—

Cuadro 2: Tabla de Parsing Predictivo LL(1) ($M[\text{No-Terminal}, \text{Lookahead}]$).

4 Esquema de Traducción Dirigida por Sintaxis (SDT) y Tabla de Símbolos

En cumplimiento con la fórmula teórica CFG + Semantic Rules $\Rightarrow$ SDT, cada nodo del *Parse Tree* posee los atributos sintetizados `.type` (tipo de dato inferido) y `.val` (valor o lexema asociado). Una vez que el motor LL(1) concluye la construcción del árbol, el módulo `sdt/analyzer.py` ejecuta un recorrido ascendente (*bottom-up traversal*) evaluando las reglas de la Tabla 3.

Regla Gramatical (CFG)	Acción Semántica Asociada (Regla SDT)
1. $S \rightarrow$ Keyword id $= E\ S_{\text{end}}$	`{ if Lookup(id.lexeme).type != "TBD": error("Redeclaración"); if Keyword.type != E.type: error("Incompatibilidad de tipos"); else: Set(id.lexeme, Type=Keyword.type, Value=E.val) }`
2. $S \rightarrow$ id $= E\ S_{\text{end}}$	`{ entry = Lookup(id.lexeme); if entry.type == "TBD": error("Variable no declarada"); if entry.type != E.type: error("Incompatibilidad de tipos"); else: Set(id.lexeme, LineOfUsage=id.line) }`
3. $E \rightarrow T\ E'$	`{ if E'.type == "void": E.type = T.type elif T.type == E'.type: E.type = T.type else: E.type = "type_error"; error("Operación inválida en E") }`
4. $E' \rightarrow +\ T\ E'_1$	`{ if E'1.type == "void" or T.type == E'1.type: E'.type = T.type else: E'.type = "type_error" }`
5. $E' \rightarrow \epsilon$ / $T' \rightarrow \epsilon$	`{ E'.type = "void" }` / `{ T'.type = "void" }`
6. $T \rightarrow F\ T'$	`{ if T'.type == "void": T.type = F.type elif F.type == T'.type: T.type = F.type else: T.type = "type_error"; error("Operación inválida en T") }`
7. $F \rightarrow$ Constants	`{ F.type = "int"; F.val = Constants.lexeme }`
8. $F \rightarrow$ Literals	`{ F.type = infer_literal_type(Literals.lexeme); F.val = Literals.lexeme }`
9. $F \rightarrow$ Identifier	`{ entry = Lookup(Identifier.lexeme); if entry.type == "TBD": F.type = "type_error"; error("No declarada") else: F.type = entry.type; Set(Identifier.lexeme, LineOfUsage=line) }`

Cuadro 3: Esquema SDT para verificación de tipos, alcance y actualización de Tabla de Símbolos.

4.1 Actualización de la Tabla de Símbolos

Durante la fase léxica, el escáner registra el identificador `$x` con atributos pendientes (TBD) mediante `Insert()`. Cuando el verificador SDT valida exitosamente la regla $S \rightarrow$ Keyword Identifier $= E\ S_{\text{end}}$ sobre la entrada `int $x=10`, invoca la operación `Set()` actualizando la entrada como se muestra en la Tabla 4.

Name	Type	Size	Dimension	Line of Decl.	Line of Usage	Address
`$x` *(Fase Léxica)*	TBD	TBD	TBD	1	TBD	TBD
`$x` *(Tras SDT)*	int	4	0	1	1	0x1004

Cuadro 4: Evolución del registro en la Tabla de Símbolos entre el Lexer y el verificador SDT.

5 Trazas Teóricas y Argumentación de Salidas Esperadas

Las especificaciones del proyecto requieren presentar los argumentos teóricos para cada una de las tres posibles salidas del sistema.

5.1 Caso 1: Éxito Sintáctico y Semántico (`int $x=10`)

Entrada: `int $x=10` (en línea 1).
Flujo de Tokens del Lexer: <Keyword, "int">, <Identifier, "$x">, <Operator, "=">, <Constants, "10">, <Special_Characters, "$">.

Step	Stack (Base → Tope)	Input Buffer (i/p)	Action (Tabla LL(1) / Match)
1	`$ S`	`int $x = 10 $`	$S \rightarrow$ Keyword Identifier $= E\ S_{\text{end}}$
2	`$ S_end E = id Keyword`	`int $x = 10 $`	`match Keyword ("int")`
3	`$ S_end E = id`	`$x = 10 $`	`match Identifier ("$x")`
4	`$ S_end E =`	`= 10 $`	`match Operator ("=")`
5	`$ S_end E`	`10 $`	$E \rightarrow T\ E'$
6	`$ S_end E' T`	`10 $`	$T \rightarrow F\ T'$
7	`$ S_end E' T' F`	`10 $`	$F \rightarrow$ Constants
8	`$ S_end E' T' Constants`	`10 $`	`match Constants ("10")`
9	`$ S_end E' T'`	`$`	$T' \rightarrow \epsilon$
10	`$ S_end E'`	`$`	$E' \rightarrow \epsilon$
11	`$ S_end`	`$`	$S_{\text{end}} \rightarrow \epsilon$
12	`$`	`$`	**`Parsing Success!`**

Cuadro 5: Traza en pila del Parser LL(1) para el ejemplo obligatorio `int $x=10`.

Evaluación SDT: En el paso de recorrido del árbol, la hoja `Constants("10")` sintetiza `F.type = "int"` $\Rightarrow$ `T.type = "int"` $\Rightarrow$ `E.type = "int"`. En la raíz S, se verifica que `$x` tiene `Type == "TBD"` (no ha sido declarada previamente) y que `Keyword.type ("int") == E.type ("int")`. Al cumplirse ambos predicados, se actualiza la Tabla de Símbolos y se imprime:

```
Parsing Success!
SDT Verified!
```

5.2 Caso 2: Éxito Sintáctico pero Error Semántico (SDT `error...`)

Entrada: `int $x="diez"` (o bien `int $x=$no_declarada + 10` en la línea 1).
Argumento Teórico: Sintácticamente, la producción $F \rightarrow$ Literals permite derivar una cadena de texto como factor válido dentro de una expresión E, por lo que la pila LL(1) llega al estado de aceptación $ sin contratiempos e imprime `Parsing Success!`. Sin embargo, durante el recorrido posorden del árbol, el nodo F sintetiza `F.type = "string"` $\Rightarrow$ `E.type = "string"`. Al ejecutar la regla semántica en la raíz S, el predicado lógico Keyword.type $==$ E.type evalúa `"int"` $==$ `"string"` $\Rightarrow$ False, disparando el fallo semántico en la línea correspondiente:

```
Parsing Success!
SDT error...
error in line 1
```

5.3 Caso 3: Error Sintáctico y Error Semántico (`Parsing error...`)

Entrada: `int $x = * 10` (en la línea 1).
Argumento Teórico: Tras hacer *match* de `int`, `$x` y `=`, el tope de la pila contiene el no-terminal E y el símbolo de preanálisis (*lookahead*) en el búfer es el operador `*`. Al consultar la Tabla de Parsing LL(1) en la celda $M[E, *]$, se encuentra vacía dado que $* \notin \text{FIRST}(E) = \{(, \text{Identifier}, \text{Constants}, \text{Literals}\}$. El motor aborta la construcción del *Parse Tree*; al no existir un árbol sintáctico válido, la fase SDT tampoco puede verificarse:

```
Parsing error...
SDT error...
error in line 1
```

6 Guía de Implementación para el Desarrollador (Python)

Restricción de Implementación: Para obtener los puntos extra por aplicar únicamente la teoría vista en clase, **queda prohibido** usar bibliotecas externas de parsing o evaluar las expresiones con la función nativa `eval()` de Python. El desarrollador debe implementar la estructura de nodos, la pila explícita LL(1) y el evaluador recursivo de atributos SDT como se detalla a continuación.

6.1 Módulo 1: Nodos del Árbol Sintáctico (`models/ast_node.py`)

```
class ASTNode:
    def __init__(self, symbol, token=None):
        self.symbol = symbol          # Simbolo gramatical (ej. 'S', 'E', 'F', 'Keyword
        self.token = token            # Objeto Token asociado (si es hoja terminal)
        self.children = []            # Lista de nodos hijos en orden de derivacion
        self.data_type = "void"       # Atributo sintetizado para el SDT
        self.line = token.line if token else 1

    def add_child(self, child_node):
        self.children.append(child_node)
```

6.2 Módulo 2: Motor Predictivo LL(1) con Pila (`parser/ll1_parser.py`)

```
from models.ast_node import ASTNode
from models.token import Token

class LL1Parser:
    def __init__(self, parsing_table, terminals):
        self.table = parsing_table    # Diccionario M[(NoTerminal, Lookahead)] -> lista
        self.terminals = terminals

    def parse(self, tokens):
        # Agregar token centinela '$' al final del bufer de entrada
        last_line = tokens[-1].line if tokens else 1
        input_buffer = list(tokens) + [Token("Special_Characters", "$", last_line)]

        root = ASTNode("S")
```

```
        end_node = ASTNode("$")
        stack = [end_node, root]      # Pila explicita inicializada con [$ S]
        idx = 0

        while len(stack) > 0:
            top_node = stack[-1]
            current_token = input_buffer[idx]
            lookahead = self._map_lookahead(current_token)

            # Caso 1: El tope de la pila es '$' y la entrada es '$' -> Exito
            if top_node.symbol == "$" and lookahead == "$":
                stack.pop()
                return True, root, None

            # Caso 2: El tope de la pila es un Terminal
            if top_node.symbol in self.terminals:
                if top_node.symbol == lookahead:
                    top_node.token = current_token
                    top_node.line = current_token.line
                    stack.pop()
                    idx += 1          # Match exitoso: avanza puntero del bufer
                else:
                    return False, None, current_token.line
            # Caso 3: El tope de la pila es un No-Terminal
            else:
                production = self.table.get((top_node.symbol, lookahead))
                if production is None:
                    return False, None, current_token.line  # Celda vacia -> Error

                stack.pop()
                # Si no es una produccion epsilon, crear hijos y apilar en orden invers
                if production != ["EPSILON"]:
                    child_nodes = [ASTNode(sym) for sym in production]
                    for child in child_nodes:
                        top_node.add_child(child)
                    for child in reversed(child_nodes):
                        stack.append(child)

        return False, None, last_line

    def _map_lookahead(self, token):
        if token.lexeme in ["=", "+", "-", "*", "/", "(", ")", ";", "$"]:
            return token.lexeme
        return token.category  # 'Keyword', 'Identifier', 'Constants', 'Literals'
```

6.3 Módulo 3: Verificador Semántico SDT (`sdt/analyzer.py`)

```
class SDTAnalyzer:
    def __init__(self, symbol_table):
        self.symtab = symbol_table
        self.error_line = None
```

```
def verify(self, root_node):
    ok = self._traverse(root_node)
    return ok, self.error_line

def _traverse(self, node):
    # 1. Recorrer hijos primero (Post-order / Bottom-Up attribute synthesis)
    for child in node.children:
        if not self._traverse(child):
            return False

    # 2. Evaluar reglas semanticas segun el simbolo del nodo
    if node.symbol == "F":
        first_child = node.children[0]
        if first_child.symbol == "Constants":
            node.data_type = "int"
        elif first_child.symbol == "Literals":
            lex = first_child.token.lexeme
            node.data_type = "bool" if lex in ["true", "false"] else "string"
        elif first_child.symbol == "Identifier":
            entry = self.symtab.lookup(first_child.token.lexeme)
            if entry is None or entry.type == "TBD":
                self.error_line = first_child.line
                return False
            node.data_type = entry.type
        elif first_child.symbol == "(":
            node.data_type = node.children[1].data_type

    elif node.symbol in ["T'", "E'"]:
        if len(node.children) == 0:
            node.data_type = "void"
        else:
            op_operand_type = node.children[1].data_type
            rest_type = node.children[2].data_type
            if rest_type == "void" or op_operand_type == rest_type:
                node.data_type = op_operand_type
            else:
                self.error_line = node.children[0].line
                return False

    elif node.symbol in ["T", "E"]:
        left_type = node.children[0].data_type
        right_type = node.children[1].data_type
        if right_type == "void" or left_type == right_type:
            node.data_type = left_type
        else:
            self.error_line = node.children[0].line
            return False

    elif node.symbol == "S":
        if node.children[0].symbol == "Keyword":
            declared_type = node.children[0].token.lexeme
            id_tok = node.children[1].token
            expr_type = node.children[3].data_type
```

```
                entry = self.symtab.lookup(id_tok.lexeme)
                if entry and entry.type != "TBD":
                    self.error_line = id_tok.line
                    return False
                if declared_type != expr_type:
                    self.error_line = id_tok.line
                    return False
                self.symtab.set(id_tok.lexeme, data_type=declared_type, line_usage=id_t

        return True
```

6.4 Módulo 4: Orquestador de Salida Oficial (`main.py`)

```
def run_compiler(source_code, lexer, parser, sdt_analyzer):
    tokens = lexer.scan(source_code)
    parse_ok, parse_tree, syntax_err_line = parser.parse(tokens)

    if not parse_ok:
        print("Parsing error...")
        print("SDT error...")
        print(f"error in line {syntax_err_line}")
        return

    print("Parsing Success!")
    sdt_ok, sdt_err_line = sdt_analyzer.verify(parse_tree)

    if sdt_ok:
        print("SDT Verified!")
    else:
        print("SDT error...")
        print(f"error in line {sdt_err_line}")
```